

Guía completa para un agente de vibe coding: Prototipo jugable de “Gangsters: Organized Crime” en Godot 4 + GDScript + Cursor + Git

Versión 1.0 – Enfoque solo-desarrollador + AI

Objetivo: Crear una demo jugable lo más fiel posible al espíritu del original de 1998 (Hothouse Creations / Eidos), pero empezando **solo con un prototipo mínimo viable**. Tú trabajas solo con ayuda de AI (Cursor principalmente). El libro está diseñado para que un agente de vibe coding (o tú mismo siguiendo el flujo) pueda ejecutarlo paso a paso sin perder el foco.

Capítulo 1. Visión y alcance del prototipo

El original combina:

- Planificación por turnos (órdenes a hoods)
- Ejecución en tiempo real (“semana de trabajo”)
- Mapa isométrico de ciudad
- Hoods con atributos
- Extorsión, reclutamiento, negocios, rivales, heat/policía
- Tres condiciones de victoria

Prototipo (Demo jugable) – Alcance estricto (2-4 semanas de trabajo realista solo + AI):

- Una sola ciudad pequeña procedural o predefinida (8×8 o 10×10 bloques).
- Vista isométrica 2.5D simple.
- 1 jugador (tú) + 1-2 bandas rivales muy simples.
- 4-8 hoods propios + algunos rivales.
- Atributos básicos de hood: Inteligencia, Intimidación, Fuerza, Organización (0-5 estrellas).
- Fase de Planificación: asignar órdenes simples (Extorsionar, Reclutar, Recoger protección, Moverse).
- Fase Real-time: los hoods ejecutan durante un timer de 30-60 segundos (o “día” acelerado).
- Sistema de dinero + protección semanal.
- 1-2 tipos de edificios (negocio extorsionable + sede de banda).

- Condición de victoria simple: controlar X bloques o acumular Y dinero en Z semanas.
- Sin abogado, contable, FBI completo, armas complejas, multijugador ni FMV.
- UI funcional pero fea (se embellece después).

Todo lo demás (más atributos, 5000 ciudadanos, 3 rivales inteligentes, alcaldía, etc.) se deja para post-prototipo.

El prototipo debe sentirse “Gangsters” en el loop principal: planificas → ves cómo se jode o funciona → ajustas.

Capítulo 2. Stack técnico y flujo de trabajo

- **Motor:** Godot 4.3+ (o la última estable).
- **Lenguaje:** GDScript 100 %.
- **Editor AI:** Cursor (con Claude 3.5/4 o GPT-4o / o el modelo que tengas).
- **Control de versiones:** Git + GitHub (repositorio privado o público).
- **Gestión de tareas:** Markdown simple (TODO.md + CHANGELOG.md) o Notion/Linear si prefieres.
- **Arte temporal:** Placeholders generados con AI (Midjourney / Flux / Leonardo) o assets gratuitos isométricos (Kenney, OpenGameArt). Luego se reemplazan.
- **Audio:** Placeholders o sonidos libres. Música de menú simple más adelante.

Regla de oro del vibe coding solo + AI:

1. Nunca escribas código largo a mano si puedes describirlo bien a Cursor.
 2. Siempre pide a Cursor que explique el “por qué” y que genere tests mentales o asserts.
 3. Commits pequeños y frecuentes.
 4. Cada sesión termina con un commit + nota en CHANGELOG.
-

Capítulo 3. Setup inicial del proyecto (Día 0-1)

1. Crea el repositorio:

```
Bash
```

```
mkdir gangsters-proto
cd gangsters-proto
git init
```

1. Crea el proyecto Godot 4:

- Nuevo proyecto → “GangstersProto”
- Configura resolución base 1280×720 o 1920×1080.
- Añade carpetas:

```
text
res://
├─ scenes/
├─ scripts/
│   ├─ core/
│   ├─ entities/
│   ├─ systems/
│   └─ ui/
├─ data/
├─ assets/
│   ├─ tiles/
│   ├─ characters/
│   └─ ui/
├─ autoload/
└─ docs/
```

1. Crea docs/VISION.md y pega el Capítulo 1.
2. Crea TODO.md con las fases de este libro.
3. Primer commit: “Initial project structure + vision”.

Prompt Cursor típico de arranque:

```
“Crea la estructura de carpetas y un Autoload llamado GameManager que tenga estados:
PLANNING, EXECUTION, WEEK_END. Añade un simple state machine. Explica cada
línea.”
```

Capítulo 4. Arquitectura core (el corazón del prototipo)

Autoloads recomendados:

- GameManager → estado global, semana actual, dinero, heat simple.

- DataManager → carga JSON de hoods, edificios, órdenes.
- EventBus → señales globales (hood_died, order_completed, money_changed...).
- MapManager → grid isométrico.

Entidades principales (scripts):

- Hood.gd (CharacterBody2D o Node2D + datos)
- Building.gd
- Order.gd (Resource)
- Gang.gd (Resource o Node)
- CityBlock.gd

Datos en JSON / Resources:

```
gdscript

# Ejemplo HoodResource
class_name HoodResource extends Resource
@export var id: String
@export var name: String
@export var intelligence: int = 3
@export var intimidation: int = 3
@export var strength: int = 2
@export var organization: int = 2
@export var loyalty: float = 1.0
@export var status: String = "idle" # idle, executing, dead, jailed
```

Fases:

1. PLANNING → UI de órdenes + mapa interactivo.
2. EXECUTION → Timer + pathfinding simple + resolución de órdenes.
3. WEEK_END → Reportes, dinero, heat, posible reclutamiento, check victoria.

Capítulo 5. Mapa isométrico mínimo

Usa TileMapLayer (Godot 4) o un sistema de grid propio con Node2D.

- Grid 8×8 o 10×10.
- Cada celda puede ser: Calle, Negocio, Sede, Vacío.

- Conversión iso:

```
gdscript

func grid_to_iso(grid_pos: Vector2i) -> Vector2:
    return Vector2((grid_pos.x - grid_pos.y) * tile_width/2, (grid_pos.x +
grid_pos.y) * tile_height/2)
```

Prompt Cursor:

“Implementa un MapManager que genere un grid 10x10 isométrico. Cada celda es un CityBlock con tipo y dueño (jugador/rival/neutral). Añade click para seleccionar bloque y mostrar info.”

Añade cámara ortográfica con zoom y pan.

Capítulo 6. Sistema de Hoods y Órdenes

Hood:

- Tiene posición en grid.
- Tiene lista de órdenes pendientes.
- En EXECUTION se mueve con AStarGrid2D o pathfinding simple.

Order (Resource):

```
gdscript

class_name Order extends Resource
enum Type { EXTORT, COLLECT, RECRUIT, MOVE, ATTACK }
@export var type: Type
@export var target_block: Vector2i
@export var assigned_hoods: Array[Hood]
@export var success_chance: float # calculado por atributos
```

En PLANNING el jugador arrastra hoods o usa UI para asignar.

En EXECUTION:

- Cada frame o cada 0.5 s se avanza el progreso de la orden.
- Al terminar se resuelve con dados + atributos (Inteligencia afecta éxito, Intimidación afecta cantidad de dinero, etc.).

Prompt Cursor clave:

*“Crea el sistema de resolución de órdenes. Para EXTORT: probabilidad base = inteligencia * 0.15 + intimidation * 0.1. Si éxito, genera dinero y marca el bloque como controlado. Si falla, aumenta heat. Incluye feedback visual simple.”*

Capítulo 7. Loop de juego completo del prototipo

1. Inicio de semana → PLANNING.
2. Jugador asigna órdenes (máximo 1-2 por hood).
3. Botón “Iniciar Semana” → EXECUTION (timer 45 s o hasta que todas las órdenes terminen).
4. Durante EXECUTION: hoods se mueven, se ven iconos de acción, posible conflicto simple con rival.
5. Fin → WEEK_END:
 - Cobrar protecciones
 - Actualizar lealtad / desertión simple
 - Generar reporte textual (“Tu hood Big Tony falló la extorsión en el bar de la esquina”)
 - Check victoria/derrota
6. Siguiendo semana.

Añade un simple rival AI: cada semana el rival intenta extorsionar 1-2 bloques neutrales o atacar uno tuyo con probabilidad baja.

Capítulo 8. UI mínima viable

- Panel izquierdo: lista de hoods (nombre + estrellas + estado).
- Panel derecho: info del bloque seleccionado + botones de órdenes.
- Barra superior: Dinero | Semana | Heat | Botón “Ejecutar Semana”.
- Overlay de reporte al final de semana (RichTextLabel).
- Mapa clickeable.

Usa Control nodes + Theme simple. Más adelante se hace estilo 1920s.

Capítulo 9. Flujo de trabajo diario con Cursor + Git

Plantilla de sesión:

1. Abre Cursor en el proyecto.
2. Lee TODO.md y elige la siguiente tarea pequeña.
3. Prompt estructurado:

text

Contexto: Estamos en el prototipo de Gangsters.

Archivo actual: scripts/systems/OrderResolver.gd

Objetivo: ...

Restricciones: Solo GDScript, Godot 4.3, no añadir dependencias.

Genera el código + explicación + posibles edge cases.

4. Revisa, prueba en el editor, ajusta con follow-ups.
5. `git add . && git commit -m "feat: order resolution for EXTORT"`
6. Actualiza TODO.md y CHANGELOG.md.

Reglas anti-caos:

- Nunca dejes Cursor generar más de 150-200 líneas de una vez.
- Después de cada feature grande pide: “Refactorea esto para que sea más limpio y añade comentarios de diseño”.
- Mantén un `docs/DECISIONS.md` con las decisiones importantes (por qué usaste Resource en vez de Node, etc.).

Capítulo 10. Roadmap del prototipo (orden recomendado)

Semana 1:

- Setup + GameManager + estados
- Grid isométrico + click
- Hood básico + movimiento

Semana 2:

- Sistema de órdenes + UI de asignación

- Resolución de EXTORT y COLLECT
- Dinero y control de bloques

Semana 3:

- Rival AI mínimo
- WEEK_END + reportes
- Condición de victoria
- Polish básico (feedback, sonidos placeholder)

Criterio de “Demo jugable terminada”:

Puedes empezar una partida, planificar 3-4 semanas, ver hoods moverse y extorsionar, ganar o perder de forma clara, y que se sienta el loop del original.

Capítulo 11. Después del prototipo (visión a medio plazo)

Una vez la demo sea jugable y divertida:

1. Más atributos y tipos de órdenes (incendio, asesinato, soborno simple).
2. Mejor IA rival y múltiples bandas.
3. Ciudad más grande + generación procedural.
4. Sistema de lealtad / desertión / promoción a teniente.
5. Heat y policía básica.
6. UI estilo 1920s + arte propio.
7. Guardado / carga.
8. Condiciones de victoria del original (Go Straight, Alcalde, Dominación).

Mantén siempre el mismo loop de vibe coding: scope pequeño → Cursor → prueba → commit.

Capítulo 12. Recursos y referencias constantes

- Manual y análisis del original (Hardcore Gaming 101 + Wikipedia + MobyGames).
- Godot Docs (especialmente TileMap, AStarGrid2D, Resources, Signals).
- Este documento como fuente de verdad del alcance.
- Carpeta docs/original_notes.md donde copies mecánicas clave del juego de 1998.

Cierre

Este libro es tu brújula. El prototipo no tiene que ser bonito ni completo: tiene que **sentirse** Gangsters en el loop de planificación → ejecución → consecuencias.

Empieza hoy mismo con el Capítulo 3.

Cada commit es progreso.

Cada prompt bien escrito a Cursor es una hora ahorrada.

Cuando termines la demo jugable, vuelve aquí y escribimos el “Libro 2: De prototipo a juego completo”.

¿Listo para el primer commit?

Abre Cursor y di:

“Vamos a construir el prototipo de Gangsters. Empieza por el GameManager con state machine según la guía.”

Éxito. cabo.